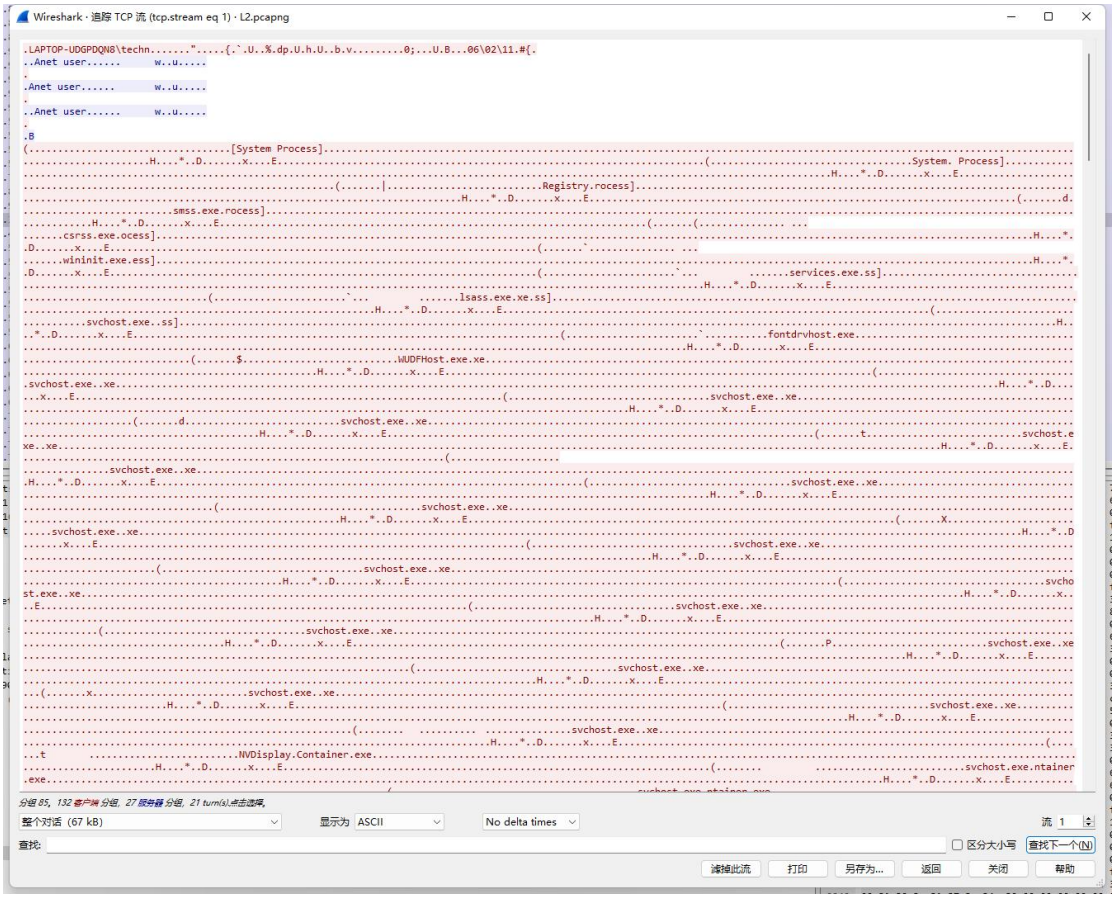
Reverse

Malware

本题的文件有两个，除了 pcap 的流量包外，还包含了木马文件 exe。当拿到此题的感觉是，flag 就在丢失的文件里，需要逆向 exe 去解密 pacp 中的流量。

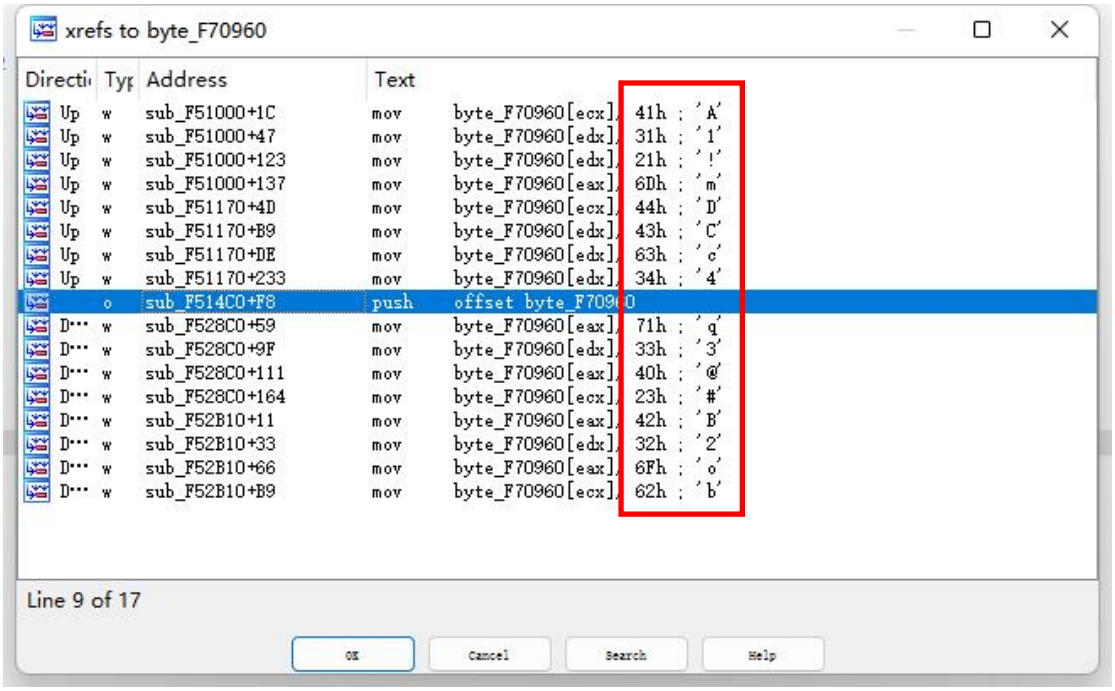
首先在 wireshark 中观察 pcap 流量包，发现都是 TCP 流量，进行流追踪可以看到有一个 TCP 流巨长，有很多包，不过实际上这个流中并没有我们要找的文件流量，具体会在后面分析。



然后就是将木马文件放入 IDA 进行静态分析，我在观察函数调用链的时候看到一个函数里有 Win32 CryptoAPI 里的 CryptImportKey，其中传进去的 pbData 指向 .rdata: 00F68330，内容是这 16 个字节：98 0A 81 98 E5 3A 86 BA 29 CF F1 62 22 7C 55 EA，根据分析可以知道，这是后续加密所使用的 IV。

```
.rdata:00F68330 pbData          db 98h
.rdata:00F68331                 db  0Ah
.rdata:00F68332                 db  81h
.rdata:00F68333                 db  98h
.rdata:00F68334                 db 0E5h
.rdata:00F68335                 db  3Ah ; :
.rdata:00F68336                 db  86h
.rdata:00F68337                 db 0BAh
.rdata:00F68338                 db  29h ; )
.rdata:00F68339                 db 0CFh
.rdata:00F6833A                 db 0F1h
.rdata:00F6833B                 db  62h ; b
.rdata:00F6833C                 db  22h ; "
.rdata:00F6833D                 db  7Ch ; |
.rdata:00F6833E                 db  55h ; U
.rdata:00F6833F                 db 0EAh
.rdata:00F68340 ; const CHAR szProvider[]
```

而密码部分，通过查看调用可以发现，在不同的位置进行分别初始化，将他们分别找到后对应 ACSII，得到的密码为 Am!1Bob2DCc4@#q3。



通过静态分析木马文件的各函数，在 sub_F52820 中发现 0x800C，它是 CryptoAPI 哈希算法 CALG_SHA_256 的标识符，表示该木马使用 SHA-256 算法。

```
__m128i *__cdecl sub_F52820(HCRYPTPROV hProv, BYTE *pbData)
{
  char *v2; // eax
  __m128i v4[4]; // [esp+0h] [ebp-48h] BYREF
  DWORD pdwDataLen; // [esp+40h] [ebp-8h] BYREF
  HCRYPTHASH phHash; // [esp+44h] [ebp-4h] BYREF

  sub_F56130(v4, 0, 0x40u);
  pdwDataLen = 32;
  if ( !CryptCreateHash(hProv, 0x800Cu, 0, 0, &phHash) )
    sub_F599C1(0);
  v2 = sub_F59670(pbData);
  if ( !CryptHashData(phHash, pbData, (DWORD)v2, 0) )
    sub_F599C1(0);
  if ( !CryptGetHashParam(phHash, 2u, (BYTE *)v4, &pdwDataLen, 0) )
    sub_F599C1(0);
  return v4;
}
```

此外，它把 pbData 这个明文字符串，做了一次 SHA-256，得到 32 字节结果，并把这 32 字节当作对称加密密钥返回。

根据后续分析 sub_F52620 函数时，发现 v5 = 26126（0x660E）。经过网上搜索发现，在 Windows 加密 API 中，用 CALG_AES_128 这个宏表示 AES-128 算法，其值定义为 0x660E，所以参与加密的 AES key = SHA256(password)的前 16 个字节。

```
v18 = sub_F596FB(v25);
pbData[0] = 8;
pbData[1] = 2;
v4 = 0;
v5 = 26126;
v6 = dword_F70000;
sub_F56130(v7, 0, 0x20u);
```

下面的函数是比较重要的部分，第一次分析的时候就是没有仔细分析这部分导致我选错了 TCP 流的数据包。这段代码主要有两点需要注意：

1.我之前在看到 sub_F51A10(s, buf, len, 0, 0)，通过 socket 发送 buf 里的数据，但是这里却传的是(s, 0, 0, 7, 0)，这也就是说 sub_ F51A10 不仅仅是 send()，而是一个协议封装函数。这里很可能是向服务器发送一个 flag=7 的控制信号，表示进入数据传输阶段，只有这个步骤成功，才继续执行后面操作。

2.sub_F513E0((int)v3, byte_F70988)它把 v3 传给 sub_ F513E0，并且还给了一个全局的 byte_F70988。sub_ F513E0 返回了一个指针，这个指针被当成 const void *拷贝到 v4，并且 sizeof(v4)是 184 字节。所以可以推测：sub_ F513E0 负责组装一段 184 字节的文件描述信息。

```
DWORD __stdcall sub_F530C0(LPVOID lpThreadParameter)
{
  int v2[46]; // [esp-B8h] [ebp-2F4h] BYREF
  int v3[46]; // [esp+8h] [ebp-234h] BYREF
  char v4[184]; // [esp+C0h] [ebp-17Ch] BYREF
  char buf[184]; // [esp+178h] [ebp-C4h] BYREF
  int v6; // [esp+230h] [ebp-Ch]
  int i; // [esp+234h] [ebp-8h]
  SOCKET s; // [esp+238h] [ebp-4h] BYREF

  v6 = 0;
  if ( sub_F518D0(&s) )
  {
    for ( i = 0; i < 3 && !sub_F519B0(s, cp, dword_F70018); ++i )
      ;
    if ( s != -1 && sub_F51A10(s, 0, 0, 7, 0) )
    {
      qmemcpy(v4, (const void *)sub_F513E0((int)v3, byte_F70988), sizeof(v4));
      qmemcpy(buf, v4, sizeof(buf));
      sub_F51A10(s, buf, 184, 0, 0);
      qmemcpy(v2, buf, sizeof(v2));
      sub_F514C0(
```

Wireshark 过滤数据流时，应筛选出拼接后 payload 非空、且首字节为 0x07、后续数据长度不少于 184 字节的流。只有满足这些条件的流，才可能包含正确的文件内容。经过观察，上面的那个巨长的流不符合这个要求，所以被过滤。剩余下来的 TCP 流量很少，经过观察找到了符合条件的 TCP 流如下：

Wireshark · 追踪 TCP 流 (tcp.stream eq 6) · L2.pcapng

```
00000000  07                                                .
00000001  00 00 00 00 44 3a 5c 74  65 6d 70 5c 32 2e 74 78  ....D:\t emp\2.tx
00000011  74 00 a7 76 fc fb fa 6b  c0 30 dd 00 c0 30 dd 00  t..v...k .0...0..
00000021  00 00 00 00 24 00 00 00  01 00 00 00 00 00 00 00  ....$... ........
00000031  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ........ ........
00000041  00 00 00 00 00 00 00 00  f0 d8 ff ff ff ff ff ff  ........ ........
00000051  00 00 00 00 4c fb 28 03  a4 fb 28 03 9c fe 28 03  ....L.(. ..(...(.
00000061  30 32 a8 76 80 c6 63 1e  fe ff ff ff b4 fb 28 03  02.v..c. ......(.
00000071  4f 76 a7 76 01 00 00 00  00 00 00 00 e8 fb 28 03  Ov.v.... ......(.
00000081  63 1b dd 00 20 00 00 00  14 00 00 00 28 07 00 00  c... ... ....(...
00000091  00 00 00 00 01 00 00 00  ac 44 00 00 28 a9 7f 00  ........ .D..(...
000000A1  28 a9 7f 00 20 07 00 00  01 00 00 00 28 a9 7f 00  (... ... ....(...
000000B1  00 00 00 00 00 00 00 00  ee f2 77 c3 e1 a0 b7 e4  ........ ..w.....
000000C1  d5 9d 03 e2 88 89 1d ad  00 aa b7 71 cf b9 ef 45  ........ ...q...E
000000D1  dd f4 bd 3c 0e e4 e1 b5                           ...<....
```

这段流量是 07 开头，并且文件描述信息长度为 184 字节，剩余的密文长度为 32 字节：eef277c3e1a0b7e4d59d03e288891dad00aab771cfb9ef45ddf4bd3c0ee4e1b5。

CryptoAPI 对于 AES 默认是 CBC 模式，然后只需要用脚本按照木马中嵌入的密码和 IV 解密 AES-128-CBC 即得到原始文件（flag），解密的代码实现如下：

```
1.  from Crypto.Cipher import AES
2.  import hashlib
3.
4.  cipher_hex = "eef277c3e1a0b7e4d59d03e288891dad00aab771cfb9ef45ddf4bd3c0ee4e1
    b5"
5.  cipher_bytes = bytes.fromhex(cipher_hex)
6.
```

```python
PASSWORD = b"Am!1Bob2DCc4@#q3"
IV_HEX = "98 0A 81 98 E5 3A 86 BA 29 CF F1 62 22 7C 55 EA"

key = hashlib.sha256(PASSWORD).digest()[:16]
iv = bytes.fromhex(IV_HEX.replace(" ", ""))

# Decrypt AES-128-CBC
cipher = AES.new(key, AES.MODE_CBC, iv)
pt = cipher.decrypt(cipher_bytes)
pt_unpadded = pt
txt = pt_unpadded.decode('utf-8')

print("Decrypted (text preview):")
print(txt)
```

解出的 flag 为：flag{1Easy_mA1War3_ANalys1s_KMI}

```
Decrypted (text preview):
flag{1Easy_mA1War3_ANalys1s_KMI}
```

总结：本题算是 MISC 流量分析+稍微难点的 Reverse 的缝合怪，解题的流程比较清楚，但是过程需要非常仔细，不然很容易落入出题人的陷阱。

整体思路是先解析流量包→定位恶意上报的 TCP 流→分析恶意软件中的加密算法并找到对应的密码、IV 等→解析木马发送文件的信号和文件信息的固定头长度→进一步找到加密数据→将找到的加密数据以 AES-CBC 解密并还原原始数据。